

Lecture Script

Learning Through Exploration

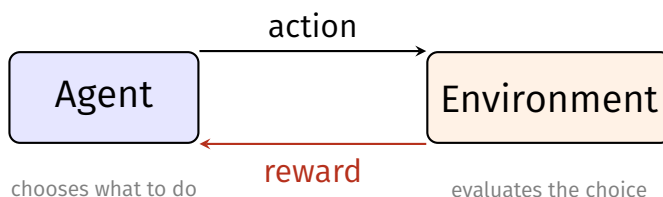
Outline

- 1 When gradients aren't enough
- 2 The RL problem
- 3 Policy gradient and REINFORCE
- 4 Case study: AlphaGo
- 5 Bridge to LLMs

1. When gradients aren't enough

In the differentiable programming lecture, we saw how to push gradients through a simulator to optimize parameters. In the SBI lectures, we handled cases where the simulator is a black box but we could still do inference by simulating forward and learning a posterior.

Now consider a different problem: you want to **optimize a decision** where the objective **cannot be differentiated through**.



Examples where the reward is non-differentiable:

- **Game outcomes:** win/loss in Go, chess — binary, no gradient
- **Experimental success:** does the drug bind? does the material conduct? — discrete yes/no from a wet lab or expensive simulation
- **Human preferences:** “is this text helpful?” — the signal in RLHF

- **Discrete choices:** which molecule to synthesize, which experiment to run next

In supervised learning, the loss $\mathcal{L}(\theta)$ is differentiable w.r.t. the model parameters because both the model output and the loss function are smooth. In RL, the agent makes a choice (possibly discrete), the environment returns a scalar reward, and we need to improve the agent's behavior from this signal alone.

Beyond the non-differentiable reward, several other things make RL harder than supervised learning:

- **Delayed reward (credit assignment).** You make 200 moves, then find out you lost. Which move was the mistake?
- **Stochastic outcomes.** You made a good move but lost anyway because the opponent got lucky. Was the move actually bad, or were you just unlucky?
- **Exploration vs. exploitation.** Do you keep doing what's worked, or try something new that might be better?
- **No fixed dataset.** The data the agent learns from depends on its own policy, which is changing as it learns.

	Supervised Learning	Reinforcement Learning
Signal	Labels (x, y)	Reward r (scalar)
Gradient path	$\nabla_{\theta} \mathcal{L}(f_{\theta}(x), y)$	no gradient through reward
Data	Fixed dataset	Agent generates its own data

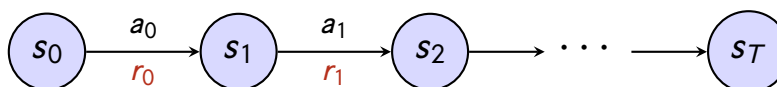
The simplest version of this — choosing which experiment to run, with no sequential structure — is the **multi-armed bandit** problem. Bayesian approaches handle it well. But in many problems, decisions are **sequential**: each choice changes the state of the world, and the reward may only come at the end of a long sequence. A Go move changes the board for all future moves; you only learn if you won after 200+ moves. An LLM generates tokens one at a time; the quality depends on all of them. These problems have **state dynamics** and **delayed reward** — this is where we need the full RL framework.

► Let's formalize this.

2. The RL problem

Markov Decision Process

The general RL setup: an agent interacts with an environment over discrete time steps.

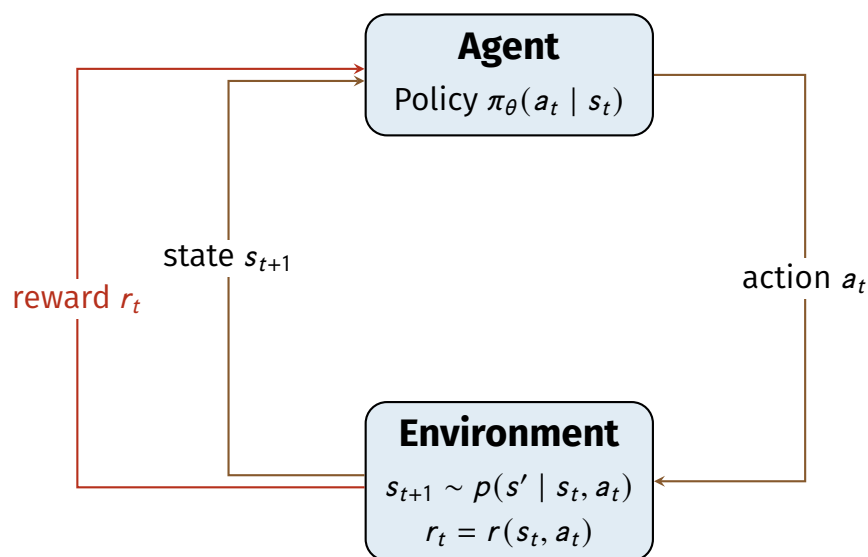


At each step t :

1. Agent observes state s_t
2. Agent chooses action $a_t \sim \pi_\theta(a \mid s)$ according to its **policy** π
3. Environment transitions to $s_{t+1} \sim p(s' \mid s_t, a_t)$ and returns reward $r_t = r(s_t, a_t)$

A **trajectory** (or episode) is a sequence $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T)$.

The full interaction loop, now with notation:



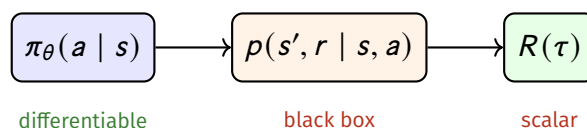
The objective

The goal is to find a policy π_θ that maximizes expected cumulative reward:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T r_t \right] = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$$

where $R(\tau) = \sum_t r_t$ is the **return** of a trajectory. The expectation is over all randomness: the policy's stochastic actions and the environment's stochastic transitions.

We want $\nabla_\theta J(\theta)$ so we can do gradient ascent. The problem: the return $R(\tau)$ depends on the environment dynamics, which we don't have a differentiable model of. The trajectory distribution $p(\tau \mid \theta)$ involves the transition probabilities $p(s' \mid s, a)$, which are a black box.



We can differentiate through π_θ (it's our neural network), but not through the environment. That's the problem.

► How do we get gradients when we can't differentiate through the reward?

3. Policy gradient and REINFORCE

The idea

We can't differentiate through the reward or the environment. But we *can* differentiate through our policy π_θ . The trick: rewrite the gradient of $J(\theta)$ so that the non-differentiable parts appear only inside an expectation, which we can estimate by sampling.

Why a stochastic policy?

A crucial detail: the policy is **stochastic**. The agent doesn't output a single action — it outputs a probability distribution $\pi_\theta(a | s)$ over actions, and then samples from it. This is what makes policy gradient work:

- **Smooth loss.** Even though individual actions are discrete, the expected return $J(\theta) = \mathbb{E}[R(\tau)]$ changes smoothly as θ changes, because we're averaging over the stochastic policy. Small changes in θ shift the distribution slightly, which shifts the expectation slightly.
- **Built-in exploration.** A stochastic policy tries different actions with different probabilities. It doesn't need a separate exploration mechanism — the randomness in sampling is the exploration.
- **Handles ambiguous observations.** If the same observation could call for different actions (e.g., the same board position might benefit from attacking or defending), a stochastic policy can assign probability to both. A deterministic policy would have to commit.

Derivation

The probability of a trajectory under policy π_θ is:

$$p(\tau | \theta) = p(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t | s_t) \cdot p(s_{t+1} | s_t, a_t)$$

The objective is:

$$J(\theta) = \mathbb{E}_{\tau \sim p(\tau | \theta)}[R(\tau)] = \int p(\tau | \theta) R(\tau) d\tau$$

Differentiate:

$$\nabla_\theta J = \int \nabla_\theta p(\tau | \theta) R(\tau) d\tau$$

Now the **log-derivative trick** (also called the REINFORCE trick). For any distribution $p(\tau | \theta)$:

$$\nabla_\theta p(\tau | \theta) = p(\tau | \theta) \cdot \nabla_\theta \log p(\tau | \theta)$$

This is just the chain rule: $\nabla \log f = \nabla f / f$, so $\nabla f = f \cdot \nabla \log f$. Substituting:

$$\nabla_{\theta} J = \int p(\tau | \theta) \cdot \nabla_{\theta} \log p(\tau | \theta) \cdot R(\tau) d\tau = \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log p(\tau | \theta) \cdot R(\tau)]$$

Now expand $\log p(\tau | \theta)$:

$$\log p(\tau | \theta) = \log p(s_0) + \sum_{t=0}^{T-1} \left[\log \pi_{\theta}(a_t | s_t) + \log p(s_{t+1} | s_t, a_t) \right]$$

When we take ∇_{θ} , the initial state $p(s_0)$ and transition dynamics $p(s' | s, a)$ don't depend on θ , so they vanish:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\left(\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) R(\tau) \right]$$

This is the **policy gradient theorem**. The environment dynamics have disappeared from the gradient. We only need:

1. The ability to **sample** trajectories by running the policy in the environment
2. The ability to **differentiate** through $\log \pi_{\theta}(a | s)$ — which is just our neural network
3. The scalar return $R(\tau)$ — which we observe, don't need to differentiate through

Intuition

What does this gradient do? Look at the update for a single trajectory:

$$\Delta \theta \propto \underbrace{\left(\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right)}_{\text{direction: make these actions more likely}} \cdot \underbrace{R(\tau)}_{\text{scale: how good was this trajectory?}}$$

$\nabla_{\theta} \log \pi_{\theta}(a | s)$ is the direction in parameter space that makes action a more likely in state s . The return $R(\tau)$ scales this:

- **High reward** trajectory: R is large $\rightarrow$ push θ to make these actions more likely
- **Low reward** trajectory: R is small (or negative) $\rightarrow$ push θ away from these actions

When we average over many trajectories, the update is largest for trajectories that are **rare but rewarding**. A trajectory the policy already produces often (high π) has a smaller $\nabla \log \pi$ — it's already likely, so there's less to gain. A trajectory the policy rarely produces (low π) has a large $\nabla \log \pi$ — if it also happens to yield high reward, the gradient pushes hard to make it more common. Conversely, common trajectories with poor rewards get pushed away.

Think of it as: the gradient does credit assignment at the trajectory level. It sifts through many rollouts and asks “which of these should I do more of?” The answer is weighted by both how good the outcome was and how surprising it was that this trajectory occurred at all.

It's trial and error formalized as gradient ascent.

The REINFORCE algorithm

Putting this into practice:

REINFORCE

1. Collect N trajectories $\{\tau^{(i)}\}$ by running π_θ in the environment
2. Compute returns $R(\tau^{(i)}) = \sum_t r_t^{(i)}$ for each
3. Estimate the gradient:

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \left(\sum_t \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)}) \right) R(\tau^{(i)})$$

4. Update: $\theta \leftarrow \theta + \alpha \hat{g}$

Variance reduction with baselines

REINFORCE as stated has **high variance**. The return $R(\tau)$ might be 100 for one trajectory and 105 for another. Both are positive, so both push the gradient in the “make these actions more likely” direction — even the worse one. The gradient is dominated by the absolute scale of R , not the relative quality.

Fix: subtract a baseline b from the return:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\left(\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) (R(\tau) - b) \right]$$

This is still an unbiased estimator of $\nabla_{\theta} J$ for any b that doesn't depend on the actions. Why? The extra term is $b \cdot \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log p(\tau | \theta)]$. This is the expected score, which is always zero for any distribution: $\mathbb{E}[\nabla_{\theta} \log p(\tau | \theta)] = \int \nabla_{\theta} p(\tau | \theta) d\tau = \nabla_{\theta} \int p(\tau | \theta) d\tau = \nabla_{\theta} 1 = 0$.

The simplest baseline: $b = \bar{R}$, the average return across the batch. Now trajectories better than average get positive reinforcement, and worse-than-average trajectories get negative reinforcement. The gradient signal is about *relative* quality.

► Let's see these ideas in action on a famous problem.

4. Case study: AlphaGo

The problem

Go is played on a 19×19 board with $\sim 10^{170}$ possible positions (chess has $\sim 10^{47}$). The game tree is far too large to search exhaustively. This maps directly onto the RL setup we just formalized:

- **State:** the current board position
- **Action:** where to place a stone
- **Policy:** a neural network $\pi_{\theta}(a | s)$ that outputs a probability over

board positions

- **Reward:** +1 for winning, -1 for losing — received only at the end of the game
- **Simulator:** the rules of Go are known exactly, so you can play as many games as you want

You have a perfect simulator and a non-differentiable reward. You can't compute $\partial(\text{win})/\partial(\text{move})$. Policy gradient is made for this.

AlphaGo (2016)

Step 1: Supervised pretraining. Train a policy network to predict expert human moves from a dataset of 30 million positions. This gives a reasonable starting policy that plays at a strong amateur level.

Step 2: Policy gradient via self-play. Who do you play against to generate training games? Yourself. The agent plays against copies of its own earlier policy. This is **self-play**: both players are the same network, so as one improves, its opponent improves too — an automatic curriculum. The return is simply $R(\tau) = +1$ or -1 depending on who won. The gradient pushes the policy toward moves from winning games and away from moves from losing games — this is REINFORCE.

Step 3: Search at test time. At each move, instead of just sampling from the policy, AlphaGo runs **Monte Carlo Tree Search (MCTS)**: it uses the policy to simulate many possible continuations of the game from each candidate move, tracks which ones lead to wins most often, and picks accordingly. The idea: the policy says which moves are *plausible*, and search figures out which are actually *good*. Search is expensive (many forward simulations per move) but produces stronger play than the raw policy alone.

AlphaGo Zero (2017)

The follow-up was even more surprising: drop the human data entirely. Start from a completely random policy and train purely by self-play. The policy generates games against itself, improves via policy gradient, and the search provides an even stronger training signal by looking ahead.

No human games, no human features, no human knowledge about Go strategy.

Result: AlphaGo Zero surpassed all previous versions — including the one that beat world champion Lee Sedol — in 40 days of training.

The general pattern

AlphaGo illustrates a recipe that shows up across science:

1. You have a **simulator** you can run cheaply (rules of Go, a physics engine, a docking code)
2. The reward is **non-differentiable** (win/loss, binding affinity, experimental success)
3. **Policy gradient** lets you optimize through this barrier — differentiate through the policy, not the reward

The same pattern applies to molecule design (reward = binding affinity), protein engineering (reward = stability), materials discovery (reward = target property from simulation), and even plasma control in fusion reactors (an RL policy controls magnetic coils to stabilize plasma, trained in a physics simulator). Anywhere you have a queryable simulator and a non-differentiable objective, policy gradient is a natural tool.

► Everything we've seen today reappears in the LLM setting.

5. Bridge to LLMs

Later in the course, we'll see that the same policy gradient framework is used to fine-tune language models from human feedback (RLHF) and verifier feedback (RLVR):

RL concept	Go	RLHF
Policy	Move predictor $\pi(a s)$	LLM $\pi_{\theta}(\text{token} \text{context})$
Action	Place a stone	Generate a token/response
Reward	Win/loss	Human preference score
Environment	Game rules	Reward model
Algorithm	REINFORCE + MCTS	Policy gradient (more later)

The log-derivative trick — getting gradients through a non-differentiable reward by differentiating the policy instead — is the same mechanism behind RLHF. More on this soon.